The make() Function in Go

Creating and initializing slices, maps, and channels

What make() does

`make()` is a built-in function that creates and **initializes** three built-in reference types: **slices, maps, and channels**. These types need internal data structures set up before they are usable (a backing array, a hash table, a channel buffer and queue), and `make` does that setup. It returns the value itself, not a pointer.

Slices

```
s := make([]int, 5)       // len 5, cap 5: [0 0 0 0 0]
t := make([]int, 0, 10)   // len 0, cap 10: [] with room for 10
```

The length is how many elements exist now, and the capacity is how much space the backing array has. Pre-setting capacity avoids repeated reallocations when you `append` in a loop:

```
result := make([]string, 0, len(items))
for _, it := range items {
    result = append(result, it.Name)
}
```

Common mistake: `make([]int, 10)` followed by `append` gives you 10 zeros and then your values. Use `make([]int, 0, 10)` if you intend to append.

Maps

```
m := make(map[string]int)         // empty, ready to use
m2 := make(map[string]int, 1000)  // size hint to reduce rehashing
m["a"] = 1
```

This is the important one. A map declared without `make` is `nil`, and **writing to a nil map panics**:

```
var m map[string]int
fmt.Println(m["x"]) // OK: reading returns 0
m["x"] = 1          // panic: assignment to entry in nil map
```

A map literal like `m := map[string]int{}` is an equivalent alternative to `make`.

Channels

```
ch := make(chan int)      // unbuffered: send blocks until a receiver is ready
buf := make(chan int, 5)  // buffered: holds up to 5 values before blocking
```

A nil channel (`var ch chan int`) blocks forever on send and receive, so channels essentially always come from `make`.

make vs new

	make	new
Works with	slices, maps, channels only	any type
Returns	the initialized value (`T`)	a pointer to a zeroed value (`*T`)
Initializes internals	yes	no, just zeroes memory

```
p := new(int)              // *int pointing to 0
m := new(map[string]int)   // *map[string]int pointing to a nil map,
                           // not usable for writes
```

So `new` is rarely what you want for these three types. For structs, people usually write `&MyStruct{}` instead of `new(MyStruct)`.

Summary

Use `make` whenever you need a slice with a preset length or capacity, a map you will write to, or a channel.